

File I/O

Bit by Bit Coding - Term 2 | Week 3

Key Concepts

- `open()` opens a file; always close it - use `with` to close automatically.
- Modes: "r" read, "w" write (overwrites), "a" append.
- `read()` reads all; `readline()` one line; `readlines()` a list of lines.
- The `csv` module reads/writes structured CSV data.
- Use `try/except` to handle missing files and other errors gracefully.

Syntax Reference

Reading & Writing Text

```
# Write (creates / overwrites)
with open("notes.txt", "w") as f:
    f.write("Hello file!")

# Append (add to end)
with open("notes.txt", "a") as f:
    f.write("\nAnother line")

# Read
with open("notes.txt", "r") as f:
    content = f.read()
    print(content)
```

Reading Line by Line

```
with open("data.txt") as f:
    for line in f:          # memory-friendly
        print(line.strip()) # strip removes \n
```

CSV with the `csv` module

```
import csv
# Read
with open("scores.csv") as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)          # row is a list

# Write
with open("out.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["name", "score"])
    writer.writerow(["Ali", 90])
```

Error Handling

```
try:
    with open("missing.txt") as f:
        data = f.read()
except FileNotFoundError:
    print("File not found!")
except Exception as e:
    print("Error:", e)
```

Common Patterns

- Use with open(...) as f: so files close even if an error occurs.
- .strip() each line to remove trailing newlines and spaces.
- Process CSV rows as lists, or use csv.DictReader for named columns.

Tips & Common Mistakes

- "w" mode ERASES the file - use "a" to keep existing content.
- Forget newline="" when writing CSV (extra blank rows on Windows).
- Reading a closed file raises an error - with handles closing for you.
- Files open from the same folder as the script unless you give a full path.